

EOB Extractor — Runbooks

Date: 2026-06-08
On-call: Monitor `eob-extractor-ops-alerts` SNS topic

RB-001: Step Functions Execution Failure

Symptoms: - CloudWatch Alarm fires on SFN `ExecutionsFailed` metric - `eob-extractor-ops-alerts` SNS notification received - SFN execution shows `FAILED` status in console

Triage:

- 1. Open the AWS Step Functions console → `eob-extraction-pipeline`
- 2. Find the failed execution (filter by `FAILED` status)
- 3. Click the execution → examine the event history
- 4. Look for the `TaskFailed` or `ExecutionFailed` event. Expand the **Error** and **Cause** fields.
- 5. Note the `correlationId` from the SFN execution name: `eob-{taskId}-{correlationId}`

Common failure states and causes:

Terminal State	Likely Cause	Action
<code>PipelineFailed</code>	Lambda threw unhandled exception	Check CloudWatch Logs (see step 6)
<code>PDFInvalid</code>	Not a failure — PDF failed validation	Check <code>validate-pdf</code> logs for <code>valid=false</code>
Execution stuck <code>RUNNING</code>	Lambda timeout	Check Lambda <code>Duration</code> metric

- 1. Query CloudWatch Logs Insights — `/aws/lambda/eob-extractor`:

```
fields @timestamp, level, event, errorName, errorMessage, correlationId, taskId
| filter correlationId = "01J3XYZ..."
| sort @timestamp asc
```

- 1. If the root cause is a transient AWS error (throttling, service unavailable), re-drive the S3 object:

```
# Re-trigger by copying the object to itself (forces a new S3 ObjectCreated event)
aws s3 cp s3://bucket-specialops/clickup/{taskId}/{file}.pdf \
  s3://bucket-specialops/clickup/{taskId}/{file}.pdf \
  --metadata-directive COPY
```

- 1. If the root cause is an `AllModelsExhaustedException` (all Bedrock models exhausted quota), check Bedrock service quotas in the AWS console and wait for daily quota reset (midnight UTC), then re-drive.
- 2. If the root cause is a bug, open a `bugfix/` branch and fix before re-driving.

RB-002: DLQ Backlog (eob-extractor-dlq)

Symptoms: - CloudWatch Alarm fires on `eob-extractor-dlq` `ApproximateNumberOfMessagesVisible` > 0 - `eob-extractor-ops-alerts` SNS notification received

Triage:

1. Read a sample message to understand the failure cause:

```
aws sqs receive-message \  
  --queue-url https://sqs.{region}.amazonaws.com/{account}/eob-extractor-dlq \  
  --max-number-of-messages 1 \  
  --attribute-names All
```

1. The message body is a stringified S3 event notification. Extract `bucket` and `key`.
2. Check `eob-trigger` CloudWatch Logs for errors around the message's `SentTimestamp`:

```
fields @timestamp, level, event, errorName, errorMessage  
| filter event like /trigger/  
| sort @timestamp desc  
| limit 20
```

1. Common DLQ causes:

Cause	Resolution
<code>STATE_MACHINE_ARN</code> env var empty	Verify env var on <code>eob-trigger</code> Lambda
S3 key does not match <code>clickup/</code> pattern	Review file naming; key is intentionally skipped, not a bug
SFN execution name conflict (duplicate)	Dedup check in trigger should prevent this; check <code>GSI-S3Key</code> lookup
Lambda cold start timeout (10s) on first invocation	Increase <code>eob-trigger</code> timeout or add provisioned concurrency

1. **Redrive messages** after root cause is fixed:

```
aws sqs start-message-move-task \  
  --source-arn arn:aws:sqs:{region}:{account}:eob-extractor-dlq \  
  --destination-arn arn:aws:sqs:{region}:{account}:eob-extractor-ingest \  
  --max-number-of-messages-per-second 1
```

1. **Purge DLQ** if messages are stale and the PDFs are no longer available or were already processed through another path:

```
aws sqs purge-queue \  
  --queue-url https://sqs.{region}.amazonaws.com/{account}/eob-extractor-dlq
```

RB-003: Bedrock Timeout / AllModelsExhaustedException

Symptoms: - `eob-classify-eob` or `eob-extract-eob` Lambda logs show `bedrock_daily_quota_exhausted` events - `AllModelsExhaustedException` in CloudWatch Logs → execution routes to `PipelineFailed` - Unusual latency spike on `eob-extract-eob` (300s timeout approaching)

Triage:

1. Check Bedrock service quotas in AWS console → Amazon Bedrock → Service Quotas :
2. Requests per minute (RPM) per model
3. Tokens per minute (TPM) per model
4. Requests per day (RPD) per model
5. Identify which model is exhausted from the log event `bedrock_daily_quota_exhausted` :

```
fields @timestamp, event, modelId
| filter event = "bedrock_daily_quota_exhausted"
| sort @timestamp desc
```

1. **If transient throttle** (`bedrock_transient_throttle_retry` logs visible): no action needed. The fallback chain handles up to 2 retries per model with exponential backoff. If retries are exhausted it falls through to the next model.
2. **If daily quota exhausted on ALL chain models:**
 3. Check Bedrock console for quota increase availability
 4. If same-day relief needed: request on-demand quota increase via AWS Support
 5. Executions that failed can be re-driven after midnight UTC (quota reset)
 6. Consider reserving Bedrock throughput (Provisioned Throughput) for the extraction model if this is recurring
7. **If Lambda timeout** (300s for extract-eob): The PDF may be unusually large. Check `s3Key` from the failed execution. If the PDF is > 10 MB, Bedrock PDF processing may exceed the timeout. No immediate fix — requires architecture change to pre-split large PDFs.

RB-004: SQS Ingest Queue Blocked

Symptoms: - `eob-extractor-ingest` `ApproximateAgeOfOldestMessage` > 5 minutes - No new SFN executions starting - `eob-trigger` Lambda invocation count drops to zero

Triage:

1. Check if `eob-trigger` Lambda is enabled:

```
aws lambda get-event-source-mapping \
  --uuid $(aws lambda list-event-source-mappings \
    --function-name eob-trigger \
    --query 'EventSourceMappings[0].UUID' --output text)
```

If `State` is `Disabled`, re-enable:

```
aws lambda update-event-source-mapping --uuid {uuid} --enabled
```

1. Check `eob-trigger` Lambda for throttling (reserved concurrency = 2):

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda \
  --metric-name Throttles \
  --dimensions Name=FunctionName,Value=eob-trigger \
  --start-time $(date -u -d '30 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 300 \
  --statistics Sum
```

1. Check if the ingest queue KMS key (`alias/eob-extractor/phi`) is accessible:

```
aws kms describe-key --key-id alias/eob-extractor/phi
```

1. Verify SFN state machine is not at maximum concurrent executions (no hard limit for STANDARD type, but check account-level quota).
2. If messages are piling up and `eob-trigger` is healthy, check if Step Functions is returning errors on `StartExecution` (duplicate execution name conflict). Review CloudWatch Logs for `trigger_sfn_s` `tarted` vs error events.

RB-005: DynamoDB PITR Restore (eob-extractions)

Use case: Data corruption, accidental delete, or disaster recovery.

Warning: A PITR restore creates a new table with a different name. It does NOT restore in-place. Plan for traffic cutover.

Steps:

1. Identify the restore point-in-time (must be within the last 35 days):

```
aws dynamodb describe-continuous-backups --table-name eob-extractions
```

1. Initiate restore:

```
aws dynamodb restore-table-to-point-in-time \
  --source-table-name eob-extractions \
  --target-table-name eob-extractions-restored-$(date +%Y%m%d%H%M) \
  --restore-date-time "2026-06-08T06:00:00Z" \
  --sse-specification-override '{"SSEEnabled":true,"SSEType":"KMS","KMSMasterKeyId":"alias/eob-extractor/phi"}'
```

1. Wait for restore to complete (can take 30-60 minutes):

```
aws dynamodb describe-table \
  --table-name eob-extractions-restored-{timestamp} \
  --query 'Table.TableStatus'
```

1. Validate the restored table data against expected record counts.
2. Update Lambda environment variables to point to the restored table:

```
aws lambda update-function-configuration \
  --function-name eob-trigger \
  --environment Variables="{STATE_MACHINE_ARN=...,TABLE_NAME=eob-extractions-restored-{timestamp}}"

aws lambda update-function-configuration \
  --function-name eob-store-result \
  --environment Variables="{TABLE_NAME=eob-extractions-restored-{timestamp},REVIEW_QUEUE_URL=...}"
```

1. Add GSIs to the restored table if they were not preserved (PITR preserves GSIs by default, but verify).
2. Add new S3 Object Lock retention on the audit bucket export if compliance records were also affected.
3. After validation, rename the restored table to the original name (not supported by DynamoDB — requires a CDK deploy with the new table name as context).

RB-006: KMS Key / Credential Rotation

KMS Key Rotation (Automatic)

Both CMKs (`alias/eob-extractor/phi` and `alias/eob-extractor/audit`) have `enableKeyRotation: true` set in CDK. AWS KMS rotates the key material annually. **No manual action required.** Existing ciphertexts remain decryptable after rotation.

To verify rotation is enabled:

```
aws kms get-key-rotation-status --key-id alias/eob-extractor/phi
aws kms get-key-rotation-status --key-id alias/eob-extractor/audit
```

Lambda IAM Role Credentials

Lambda execution roles use short-lived STS credentials managed by AWS. No rotation needed.

CDK Deploy Role (CI/CD)

If OIDC federation is used for CDK deployment (recommended): 1. Verify the GitHub Actions OIDC provider is not expired 2. Rotate the CDK bootstrap role if long-lived access keys are in use:

```
# List access keys for the CDK deploy user
aws iam list-access-keys --user-name {cdk-deploy-user}
# Create new key, update CI/CD secret, then delete old key
aws iam create-access-key --user-name {cdk-deploy-user}
aws iam delete-access-key --user-name {cdk-deploy-user} --access-key-id {old-key-id}
```

SNS Subscription Credentials (if webhooks use tokens)

Check SNS subscription endpoints for `eob-extractor-ops-alerts` and rotate webhook tokens per the consuming service's rotation process.

RB-007: Review Queue Backlog (eob-extractor-review)

Symptoms: - `eob-extractor-review` `ApproximateNumberOfMessagesVisible` growing - `REVIEW_PENDING` extractions accumulating without human action

Context: The review queue holds low-confidence (score 0.50–0.84) or insurance-mismatch extraction records that require human validation. There is **no DLQ** on this queue (Audit finding TD-005). Messages expire after 14 days.

Triage:

1. Check queue depth:

```
aws sqs get-queue-attributes \
  --queue-url https://sqs.{region}.amazonaws.com/{account}/eob-extractor-review \
  --attribute-names ApproximateNumberOfMessagesVisible,ApproximateAgeOfOldestMessage
```

1. Sample a message to identify the backlog composition:

```
aws sqs receive-message \
  --queue-url https://sqs.{region}.amazonaws.com/{account}/eob-extractor-review \
  --visibility-timeout 30 \
  --max-number-of-messages 1
```

Message body format:

```
{
  "extractionId": "01J3DEF...",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ...",
  "confidenceScore": 0.72,
  "status": "REVIEW_PENDING"
}
```

1. Query DynamoDB for pending extractions using GSI-Status:

```
aws dynamodb query \
  --table-name eob-extractions \
  --index-name GSI-Status \
  --key-condition-expression "#s = :s" \
  --expression-attribute-names '{"#s":"status"}' \
  --expression-attribute-values '{"s":{"S":"REVIEW_PENDING"}}' \
  --limit 20
```

1. Resolution options:

Scenario	Action
Normal review backlog	Notify the review team; no operational action
Consumer application crashed	Restart consumer; verify no messages lost (no DLQ — messages return to queue on visibility timeout)
Backlog > 14 days old (messages expiring)	Fix TD-005 (add DLQ); update <code>status</code> in DynamoDB to <code>FAILED</code> for expired records

Scenario	Action
False positives (documents correctly extracted but marked REVIEW_PENDING)	Review confidence threshold — current MEDIUM threshold is 0.50; consider tuning up to 0.65

1. **Escalation path for TD-005:** Add a DLQ to `eob-extractor-review` in `QueuingConstruct`. CDK change required:

```
this.reviewQueue = new sqs.Queue(this, 'ReviewQueue', {
  // ...existing props...
  deadLetterQueue: {
    maxReceiveCount: 5,
    queue: this.dlq,
  },
});
```